Breathe ESG

Tech Internship Assignment

ESG Data Ingestion & Review Platform

Design Workflow, Reasoning, and Technical Decisions

Author	Mahendra Limanpure
Institute	Indian Institute of Technology, Roorkee
Programme	B.Tech
Date	May 25, 2026
Assignment	Breathe ESG — 4-Day Prototype Challenge

This document explains every design decision made during the development of a Django + React prototype for ingesting, normalising, and auditing Scope 1, 2, and 3 emissions data from three enterprise sources.

Contents

1. Problem Statement and Context

Breathe ESG ingests emissions and activity data from client companies to help them measure, verify, and report their carbon footprint. The core challenge is not arithmetic—calculating CO_2 from a number is simple multiplication—but *data heterogeneity*: every client's data lives in a different system, with a different shape, in different units, and with different gaps.

1.1. Assignment Brief

A Project Manager has onboarded a new enterprise client (modelled as a large Indian manufacturing company) with data spread across three internal systems:

- **SAP ERP** — fuel and procurement records kept by the finance department.
- **Utility Portal** — electricity consumption data downloaded by the facilities team.
- **Corporate Travel Platform (Concur-style)** — flight, hotel, and ground-transport records managed by HR/admin.

The objective is to build a prototype web application (Django REST backend + React frontend) that:

1. Ingests data from all three sources.
2. Normalises units and computes CO_2 emissions per row.
3. Automatically flags suspicious rows.
4. Surfaces a review dashboard where an analyst can approve or reject rows.
5. Locks approved rows for auditors.

1.2. Why This Problem Matters

Under frameworks such as GRI, BRSR (SEBI mandate for top 1000 Indian listed companies), and the forthcoming CSRD, companies must publish verified emissions figures. An error in source data propagates directly into a regulatory filing. The analyst review step therefore exists not as a bureaucratic formality, but as a legally significant quality gate.

2. Scope 1, 2, and 3 — What They Mean

Before any technical decisions, it is important to understand the GHG Protocol categorisation that governs how each data row is labelled.

3. Data Source 1: SAP Fuel and Procurement

3.1. What SAP Actually Exports — Research Summary

SAP is the dominant ERP system in large Indian enterprises. It does not produce a single standardised export; instead, several mechanisms exist:

Scope	Definition	Our Data Source
Scope 1	Direct emissions from sources owned or controlled by the company (burning fuel in own vehicles, generators, furnaces).	SAP fuel records (diesel, petrol, LPG)
Scope 2	Indirect emissions from purchased electricity, steam, or heat.	Utility portal CSV (kWh consumption)
Scope 3	All other indirect emissions in the value chain (employee travel, upstream procurement, logistics).	Corporate travel platform (flights, hotels, cabs)

Table 1: GHG Protocol scope classification mapped to our three data sources.

Format	Description	Feasibility for Prototype
IDoc (Intermediate Document)	SAP's proprietary XML-like envelope used for system-to-system EDI transfer. Segments are fixed-width, nested, and require an SAP parser or middleware.	**Too complex**
OData / REST API	Available in SAP S/4HANA via Gateway. Requires OAuth credentials and knowledge of SAP entity sets.	**Needs live SAP**
BAPI	SAP's internal remote function calls. Requires SAP GUI or RFC SDK.	**Not accessible**
Flat File (CSV / XLSX)	Any SAP transaction (e.g. MB51 Material Documents, ME2M Purchase Orders) can be exported via `List` → `Export` → `Spreadsheet`. This is how most facilities and finance teams actually share data with external parties.	**Chosen**

Table 2: SAP export mechanism options and rationale for selection.

Decision: SAP Flat File (CSV)

We choose the **flat-file CSV export** from SAP transaction MB51 (Material Document List). This is the most realistic choice for an onboarding scenario: a finance officer runs the transaction, selects a date range, exports to spreadsheet, and emails or uploads the file. It requires no live SAP credentials, no middleware, and no SAP-specific SDK — all of which would be impractical in a prototype and unrealistic for an external ESG vendor to access directly.

3.2. What Realistic SAP CSV Data Looks Like

SAP field names in MB51 exports are the underlying ABAP field names, which may be in German or English depending on system configuration:

Listing 1: Realistic SAP MB51 flat file CSV (fuel procurement)

```
MANDT,BUKRS,WERKS,BLDAT,MATNR,MAKTX,MENGE,MEINS,DMBTR,WAERS
100,1000,PL01,20240115,000000000500012,Diesel EN590,500.000,L
    ,45000.00,INR
100,1000,PL02,20240120,000000000500013,Motor Spirit,300.000,L
    ,28500.00,INR
100,2000,PL01,20240201,000000000500012,Diesel EN590,50000.000,L
    ,4500000.00,INR
100,1000,PL03,20240215,000000000500014,LPG Industrial,-200.000,
   KG,18000.00,INR
100,1000,PL01,20240301,000000000500012,Diesel EN590,480.000,GAL
    ,41000.00,INR
```

3.2.1. Key Parsing Challenges and How We Handle Them

1. **Date format YYYYMMDD** (field `BLDAT`): SAP stores dates as eight-digit integers with no separators. We parse using Python's `datetime.strptime(value, '%Y%m%d')`.
2. **Unit inconsistency** (field `MEINS`): The same material can appear in litres (`L`), gallons (`GAL`), or cubic metres (`M3`) depending on the purchase order. We maintain a unit conversion table and normalise everything to litres or kilograms before computing emissions.
3. **Negative quantities**: SAP uses negative MENGE values to represent material reversals (goods returned). These are not consumption events and must be flagged.
4. **Plant codes** (`WERKS`): `PL01` means nothing without a lookup. We maintain a plant master table (`PL01 = Mumbai Factory`, `PL02 = Pune Warehouse`, etc.) populated during client onboarding.
5. **Material numbers** (`MATNR`): The 18-digit SAP material number is meaningless; we use `MAKTX` (material description) to map to our emission factor catalogue.

3.3. Why Not IDoc?

Trade-off: IDoc rejected

IDoc files require either an SAP system on the receiving end or a specialist middleware (e.g. MuleSoft, SAP PI/PO). Parsing IDoc segments manually involves handling fixed-width records, control records, and data records with segment types specific to each message type (e.g. MATMAS for materials, INVOIC for invoices). This is a multi-week integration effort with significant infrastructure cost. For an ESG data platform whose clients will not grant direct SAP system access to a third-party vendor, CSV flat-file upload is both more realistic and more secure.

4. Data Source 2: Electricity (Utility Data)

4.1. How Facilities Teams Actually Get Electricity Data

Indian facilities teams interact with electricity data in three ways:

Mode	Reality	Feasibility
PDF Bill	Utilities (MSEDCL, TATA Power, BESCOM) email a PDF invoice monthly. Contains meter reading, units consumed, tariff, and amount. Parsing requires a PDF extraction library and regex patterns specific to each utility's layout.	**Complex, skipped**
Portal CSV Export	Most major Indian utilities (MSEDCL, CESC, KSEB) offer an online portal where account holders can download usage history as CSV for a selected date range. Facilities teams routinely do this.	**Chosen**
Utility API	Green Button API (US standard) and a few progressive Indian utilities offer REST APIs. Rare in India; requires formal data-sharing agreements.	**Not realistic in India**

Table 3: Electricity data acquisition modes.

Decision: Portal CSV Export

We model the ingestion as a **CSV file upload** downloaded from the utility's self-service portal. This is what a facilities manager actually does: log in to MSEDCL or TATA Power portal, select date range, click "Download Usage History", and upload the CSV to our platform. It requires no API keys, no PDF parsing, and is a workflow the client already performs today.

4.2. What a Realistic Electricity Portal CSV Looks Like

Listing 2: Electricity usage history CSV (MSEDCL-style portal export)

```
account_number,meter_id,billing_period_start,billing_period_end
    ,
units_consumed,unit,demand_kva,tariff_category,amount_inr
ACC-001,MTR-4521,2024-01-01,2024-01-31,12450,kWh,85.5,
    Commercial-HT,187500
ACC-001,MTR-4521,2024-02-01,2024-02-29,11200,kWh,82.0,
    Commercial-HT,168000
ACC-002,MTR-7823,2024-01-15,2024-02-14,3200,kWh,24.5,Commercial
    -LT,48000
```

4.2.1. Key Parsing Challenges

1. **Billing period does not align with calendar month**: Row 3 above covers 15 January to 14 February. For monthly emissions reporting we must apportion: 17 days in January (17/31 of 3200 kWh) and 14 days in February (14/29 of 3200 kWh). This is a non-trivial but important transformation.
2. **Multiple meters per site**: A large factory may have 10+ meters (main building, warehouse, canteen, etc.) under one account. All must be summed for total Scope 2 emissions.
3. **Unit variation**: Most meters report in kWh. Some older industrial meters report in kVAh (kilovolt-ampere-hours), which differs from kWh by the power factor (typically 0.85–0.95). We flag kVAh rows for analyst review since power factor data is rarely in the CSV.

4.3. Why Not PDF Parsing?

Trade-off: PDF ingestion deliberately excluded

Every Indian utility produces a PDF bill with a completely different layout. TATA Power's PDF has a different table structure to MSEDCL's, which differs again from BESCOM's. Building a robust multi-utility PDF parser requires either: (a) custom regex per utility (months of work), or (b) a vision-based LLM extraction pipeline (operationally complex). In a 4-day prototype, attempting PDF parsing would produce fragile, untested code. The CSV portal export achieves the same result with one-tenth of the complexity, and is how facilities teams already extract this data today.

5. Data Source 3: Corporate Travel

5.1. How Travel Platforms Export Data

Platforms like Concur (SAP), Navan, and TravelPerk allow administrators to export trip reports as CSV or Excel. These are the standard formats used for expense reimbursement and now increasingly for ESG reporting.

Decision: Concur-style CSV export

We model travel ingestion as a **CSV export** from the corporate travel platform, mimicking the Concur "Trip Report" export format. This is what the admin or HR team downloads from the portal for expense reconciliation; the same file is re-used for emissions calculation. No API integration is required for the prototype.

5.2. What a Realistic Travel CSV Looks Like

Listing 3: Corporate travel expense export (Concur-style)

```
employee_id,trip_date,category,origin,destination,distance_km,
nights,class,vendor,amount_inr,currency
```

```
EMP001,2024-01-10,flight,BOM,DEL,1148,,,IndiGo,15000,INR
EMP002,2024-01-15,hotel,,Mumbai,,3,,Marriott,24000,INR
EMP003,2024-01-20,flight,BOM,XYZ,,,economy,,12000,INR
EMP004,2024-01-22,cab,,Mumbai,,,,Ola,850,INR
```

5.2.1. Key Parsing Challenges

1. **Distance not always provided**: Some records only contain origin and destination airport codes. We use the Haversine formula with a standard airport coordinates lookup (IATA database) to compute great-circle distance, then multiply by 1.09 (ICAO standard indirect routing factor).
2. **Unknown airport codes**: Row 3 above has destination `XYZ`, which is not a valid IATA code. Distance cannot be computed; the row is automatically flagged.
3. **Category drives emission factor**: A flight emits very differently from a cab. We apply different emission factors per category (see Section 6).

6. Emission Factor Calculation

6.1. No Machine Learning Required

Carbon emission calculation is deterministic multiplication using **published government emission factors**. No model training, no inference pipeline, and no AI component are needed for this step.

$$\text{CO}_2\ (\text{kg}) = \text{Quantity (normalised units)} \times \text{Emission Factor}$$

6.2. Emission Factors Used

6.3. Unit Normalisation Before Calculation

Before applying any emission factor, all quantities are converted to a standard unit:

7. Automatic Flagging Logic

7.1. Philosophy: Row-Level, Not File-Level

A critical design decision is that flagging operates at the **row level**. Uploading a 1000-row CSV does not result in a binary accept-or-reject decision on the entire file. Each row is evaluated independently and receives one of three statuses:

- **PENDING** — passes all checks; ready for analyst bulk-approval.
- **FLAGGED** — fails one or more checks; requires individual analyst review.
- **REJECTED** — analyst has reviewed and determined the row is erroneous.

This design is essential for usability: if a 1000-row SAP export has 8 suspicious rows, the analyst should review only those 8—not manually scan all 1000.

Material / Activity	Unit	kg CO_2e	Source
Scope 1 — Fuel Combustion			
Diesel	per litre	2.68	IPCC AR6 / MoEFCC
Petrol (Motor Spirit)	per litre	2.31	IPCC AR6 / MoEFCC
LPG	per kg	2.98	IPCC AR6
CNG	per kg	2.54	IPCC AR6
Scope 2 — Purchased Electricity			
India grid average	per kWh	0.82	CEA 2023 CO_2 baseline
Scope 3 — Business Travel			
Flight (economy)	per km/-pax	0.255	DEFRA 2023
Flight (business)	per km/-pax	0.357	DEFRA 2023
Hotel stay	per night	31.0	DEFRA 2023
Cab / taxi	per km	0.149	DEFRA 2023

Table 4: Emission factors used in the prototype. All values are in kg CO_2e per stated unit.

From	To	Factor	Notes
Gallons (GAL)	Litres	× 3.785	US gallon
Cubic metres (M3)	Litres	× 1000	
kVAh	kWh	× PF (default 0.9)	Flagged for review
Miles	km	× 1.609	

Table 5: Unit conversion table applied during ingestion normalisation.

7.2. The Seven Automatic Flag Checks

7.3. Analyst Workflow After Flagging

1. Analyst opens the review dashboard.
2. **Clean rows** (PENDING): reviewed briefly, approved in bulk with one click.
3. **Flagged rows**: each displayed with the specific flag reason. The analyst has three options:
   - **Approve anyway** — e.g. "Yes, the 50,000 L row is a confirmed annual bulk diesel purchase."

#	Check	Condition	Flag Reason
1	Negative quantity	`MENGE < 0`	Possible SAP reversal entry
2	Zero quantity	`MENGE == 0`	Empty / placeholder row
3	Future date	`date > today`	Likely data entry error
4	Stale date	`date < reporting_period_start`	Outside scope of report
5	Unknown unit	Unit not in conversion table	Cannot normalise
6	Statistical outlier	`quantity > mean + 3`σ per material	Possible data error or bulk purchase
7	Duplicate row	Same date + plant + material + quantity already exists	Double upload

Table 6: Automatic flag checks applied to every ingested row.

- **Reject** — row is marked REJECTED and excluded from emissions totals.
- **Edit and approve** — analyst corrects a unit or quantity, records the edit in the audit trail, then approves.

4. Once a row is APPROVED, its status is locked; no further edits are permitted without a new audit trail entry.

8. Data Model Design

8.1. Core Principles

The data model must satisfy four requirements stated in the assignment:

1. **Multi-tenancy**: multiple client companies use the same platform.
2. **Source-of-truth tracking**: every row knows which file produced it and when.
3. **Scope categorisation**: Scope 1, 2, 3 is an attribute of each row.
4. **Audit trail**: edits and approvals are logged immutably.

8.2. Primary Table: `EmissionRecord`

8.3. Supporting Tables

- **Tenant**: client company, onboarding date, reporting period.

Column	Type	Purpose
`id`	UUID	Primary key
`tenant_id`	FK → Tenant	Which client company this row belongs to
`source_type`	ENUM	`sap_fuel`, `electricity`, `travel`
`upload_batch_id`	FK → UploadBatch	Which upload produced this row
`record_date`	DATE	Normalised consumption date
`material`	VARCHAR	e.g. `diesel`, `electricity`, `flight`
`quantity_raw`	DECIMAL	Original quantity as uploaded
`unit_raw`	VARCHAR	Original unit as uploaded
`quantity_normalised`	DECIMAL	After unit conversion
`unit_normalised`	VARCHAR	Standard unit (L, kWh, km, nights)
`co2_kg`	DECIMAL	Calculated emission in kg CO_2e
`scope`	ENUM	`scope_1`, `scope_2`, `scope_3`
`status`	ENUM	`pending`, `flagged`, `approved`, `rejected`
`flag_reason`	VARCHAR	Human-readable reason for flag (nullable)
`raw_data`	JSONB	Complete original CSV row preserved verbatim
`created_at`	TIMESTAMP	When row was ingested
`is_locked`	BOOLEAN	True once approved; prevents further edits

Table 7: EmissionRecord — the central table storing every ingested data row.

- **UploadBatch**: one record per file upload — filename, upload timestamp, uploader user, source type, total rows, flagged count.
- **AuditLog**: every status change (approve, reject, edit) is appended here with timestamp, user, old value, new value. This table is insert-only; no updates or deletes.
- **PlantMaster**: maps SAP plant codes (`WERKS`) to human-readable site names per tenant.
- **EmissionFactor**: the lookup table of factors used at the time of calculation, versioned so future factor updates do not silently change historical figures.

9. Technology Stack and Deployment

9.1. Stack Decisions

9.2. Why Supabase for the Database?

Supabase provides a managed PostgreSQL instance on a free tier sufficient for thousands of rows of sample data. Django connects to it identically to any other PostgreSQL database via `psycopg2` — Supabase adds no application-layer complexity. The free tier provides

Layer	Technology	Reason
Backend	Django + DRF	Assignment requirement; mature ORM; built-in admin
Frontend	React	Assignment requirement; component model suits dashboard
Database	PostgreSQL (Supabase)	Free tier, JSONB for raw_data, reliable
CSV parsing	pandas	Handles encoding issues, type inference, missing values
Backend deploy	Render.com	Free tier, native Python/Django support
Frontend deploy	Vercel	Free tier, one-command deploy for React

Table 8: Technology choices and justifications.

500 MB of storage, far exceeding the needs of a prototype with three sample datasets.

10. What Was Deliberately Not Built

1. **PDF electricity bill parsing**: As discussed in Section 4.3, this would require utility-specific regex patterns or a vision model pipeline. The CSV portal export achieves the same outcome with a fraction of the complexity.
2. **Live SAP OData / API integration**: Connecting to a live SAP system requires RFC credentials, VPN access, and IT approval from the client — none of which are available in a prototype context. The flat-file upload models the realistic onboarding flow.
3. **Automated emissions reporting / PDF export**: Generating a formatted audit report is a valuable feature but requires significant frontend work. Given the 4-day constraint, priority was given to the ingestion and review pipeline, which is the core value proposition.

11. Open Questions for the PM

If given the opportunity to speak with the PM before building, the following ambiguities would be resolved first:

1. **Reporting period**: Is the client reporting calendar year (Jan–Dec) or financial year (Apr–Mar)? This affects how billing-period apportionment is handled for electricity.
2. **SAP plant master**: How will we receive the mapping of `WERKS` codes to site names? Is there a master data extract the client can provide at onboarding?
3. **Electricity state / grid zone**: India's grid emission factor varies by state. Which states are the client's facilities in, and should we use state-specific CEA factors or the national average?

4. **Travel class default**: When Concur export does not include cabin class for flights, should we default to economy (lower factor) or flag for review?
5. **Multi-currency**: SAP rows may carry costs in INR, USD, or EUR. Is cost relevant for emissions calculation, or is it metadata only?

12. Summary

Summary of All Design Decisions

- **SAP ingestion**: CSV flat-file upload (MB51 export) chosen over IDoc, OData, or BAPI due to realistic accessibility and simplicity.
- **Electricity ingestion**: Portal CSV download chosen over PDF parsing (too complex) and API (not available in India at scale).
- **Travel ingestion**: Concur-style CSV export; Haversine formula used for missing flight distances from IATA airport codes.
- **Emission calculation**: Deterministic multiplication using IPCC, CEA, and DEFRA published factors. No ML model required.
- **Flagging**: Row-level, not file-level. Seven automated checks. Analyst reviews only flagged rows; clean rows batch-approved.
- **Database**: PostgreSQL via Supabase free tier. JSONB column preserves raw source data verbatim for audit purposes.
- **Deployment**: Render (backend) + Vercel (frontend).